ECO SENTINEL AFRICA

Surveillance environnementale autonome

Vision SAM 3.1 • Router Muse Wonder • Agents Muse Spark
Pipeline temps réel • Dashboard • Alertes sonores

DOSSIER JURY

Projet pilote Abidjan — extensible à l'Afrique

Novembre 2026 | Python 3.12 • FastAPI • Docker • Meta API
essbony@outlook.fr • github.com/essbony/Eco-sentinel-Africa

LIVE : eco-sentinel-africa-latest.onrender.com/dashboard

SOMMAIRE

1.	Vision & Problème	3
2.	Solution & Proposition de valeur	3
3.	Fonctionnalités clés	4
4.	Architecture & Pipeline	5
5.	Stack technique	6
6.	Intelligence artificielle — Vision & Agents	7
7.	Dashboard temps réel	8
8.	Qualité, Sécurité & CI/CD	9
9.	Déploiement & Exploitation	10
10.	Résultats & Démonstration	10
11.	Limites, Éthique & Garanties	11
12.	Roadmap & Impact Afrique	11
13.	Équipe & Contact	12

Annexes : Structure du projet • API • Variables d'environnement • Liens utiles

1 Vision & Problème

En Afrique, les **dépôts sauvages** et pollutions visibles (plastiques, débris, conteneurs) prolifèrent le long des routes, marchés et lagunes. À Abidjan, les services municipaux manquent d'outils de **détection précoce** et de **preuve objective** pour intervenir vite et cibler les zones critiques.

Le coût est triple : **sanitaire** (eaux stagnantes, maladies), **environnemental** (plastiques dans la lagune Ébrié, sols pollués) et **économique** (tourisme, attractivité, temps agents). Sans surveillance continue, les dépôts se déplacent plus vite que les patrouilles.

Notre vision : un réseau de sentinelles autonomes — caméras, drones, mobiles — qui **voit, comprend et alerte en moins d'une minute**, sans chatbot, sans intervention humaine permanente. Une infrastructure sobre, réplicable ville par ville.

Le problème en chiffres (Abidjan)

- 1 200 t de déchets plastiques / jour non collectés (estimation municipale)
- 60 % des dépôts sauvages réapparaissent sous 72 h après nettoyage
- 0 outil temps réel de cartographie avant Eco Sentinel

2 Solution & Proposition de valeur

Eco Sentinel Africa est un **système autonome de surveillance environnementale** : un flux caméra entre, une alerte géolocalisée, tracée et expliquée sort — visible instantanément sur un dashboard avec **alerte sonore**.

TEMPS RÉEL	EXPLICABLE	OPÉRABLE
Détection → alerte < 1 min. Boxes & masks visibles sur image et vidéo.	5 agents A2A tracent faits / hypothèses / incertitudes. Pas de boîte noire.	1-click Docker, healthcheck, métriques, prêt Railway/Render, coût maîtrisé.

Contrairement aux chatbots généralistes, **le pipeline est fiable par construction** : ingestion découplée avec backpressure, déduplication temporelle, persistance, broadcast WebSocket et fallback déterministe si le LLM est indisponible.

3 Fonctionnalités clés

■ **Détection < 1 min** — Vision → événement en <50 ms (mock) / 2–4 s (Meta SAM 3.1). Garantie : même si l'API est vide, le fallback mock assure une réponse.

■ **Boxes & masks visibles** — Chaque détection porte une *bbox* normalisée 0–1 + un masque polygone 8 points, filtrés par NMS (IoU 0,45) et triés par score. Rendu DOM + SVG sur *camera_001* et *camera_video/live*.

■ **Alerte sonore immédiate** — Tout *trigger* récent (<15 s, non historique) déclenche un son : double bip 880 Hz + pulse rouge pour *high/critical*, bip 520 Hz sinon. Sans condition de focus — l'alerte ne rate jamais.

■ **Signal *high* précis** — Seuil 0,45 ignoré, grille 0,78 → *high* / 0,90 → *critical*, NMS anti-double-comptage, hallucination mock réduite 70 % → 14 %.

■ **Ingestion découplée** — WebSocket binaire/base64, *asyncio.Queue* 100, *sample_fps* 1,0 (backpressure : drop du plus ancien), validation 5 MB et *source_id* strict.

■ **5 agents A2A corrélés** — *classification* ■ *geo* → *research* → *analysis* → *report*, tous corrélés par *correlation_id* avec latences exposées.

Garantie affichée dans le produit : SAM détecte des objets, il ne prouve pas la toxicité chimique. Toute inférence est tracée comme hypothèse.

4 Architecture & Pipeline

Le système suit un **pipeline événementiel** clair, de la caméra au dashboard, sans couplage synchrone bloquant.

1. Ingestion	2. Vision	3. Detector	4. Router	5. Agents	6. Alertes	7. Dashboard
WS binaire/base64 Queue bornée Backpressure	SAM 3.1 (Meta) NMS 0,45 Fallback Spark	Dedup 60 s Seuil 0,5 Persistance 1	Muse Wonder Plan A2A Bus in-process	5 agents Spark Corrélés Latences	Règles + dedup Store 1000 Broadcast WS	Live + boxes Son + flash Temps réel

Persistance : *EventStore* in-memory (OrderedDict + asyncio.Lock, max 1000) remplaçable par Postgres. Observabilité : logs corrélés par *correlation_id*, métriques compteurs/latences. Sécurité : validation *source_id*, taille image, sanitization prompt.

Flux : **caméra** → **ingestion** → **SAM 3.1** → **detector** → **Router Wonder** → **[classification ■ geo]** → **research** → **analysis** → **report** → **moteur alertes** → **store + WS** → **dashboard**

5 Stack technique

Couche	Technologie	Version / Détail
Langage	Python	3.12
Manager	uv	0.12 — venv local, lock reproductible
API	FastAPI + Uvicorn[standard]	0.115 / 0.32 — OpenAPI /docs
Vision	Meta SAM 3.1 (Responses API)	sam-3.1 — OpenAI-compatible
Orchestration	Meta Muse Wonder	muse-wonder-1.0 — plan A2A
Agents	Meta Muse Spark	muse-spark-1.3 — 5 agents
Validation	Pydantic / pydantic-settings	2.9 / 2.5 — 12-factor
Qualité	Ruff / Mypy / Pytest + cov	0.17 / 2.4 / 9.1 — 28 tests, 51% cov
Sécurité	Bandit / pip-audit / Semgrep	SAST + CVE, ruff S, CI strict
Éval LLM	DeepEval	4.3 — golden set
Deploy	Docker + Compose	multi-stage, HEALTHCHECK /health

Fournisseur unique Meta (BASE_URL *https://api.meta.ai/v1*, clé *MODEL_API_KEY* prioritaire) via client *OpenAI AsyncOpenAI* — Responses + chat compatibles. Fallback *mock* déterministe (hash MD5) uniquement si clé vide, tracé *simulated:true*.

6 Intelligence artificielle — Vision & Agents

Vision SAM 3.1 : l'image est encodée base64 et envoyée à l'API Responses (*instructions* + *input_image*) avec format JSON imposé. Le prompt SAM est resserré sur les catégories *waste, plastic_bottle, container, debris, dumping*. Si SAM retourne 0 résultat légitime (ex. page Microsoft Outline), on ne fallback pas — on respecte le vide. Si l'API échoue, Spark prend le relais en chat. Chaque résultat porte *category, score, bbox, mask 8 points*, filtré (<0,45 ignoré) et dédupliqué par NMS.

Router Muse Wonder : décide du plan A2A. Par défaut : *classification* et *geo* en parallèle, puis *research* (références vérifiables, jamais inventées), *analysis* (grille *low/medium/high/critical* — 0,78→high, 0,90→critical) et *report*. Protocole *eco_sentinel.a2a.protocol* (AgentCard, A2ARequest/Response, timeout 10 s, in-process).

Garanties IA : toute inférence hors vision est marquée *hypothèse* avec incertitudes ; la détection ne prétend jamais prouver une toxicité chimique.

Agent	Rôle	Modèle
classification	Faits / hypothèses / incertitudes visuelles	Muse Spark 1.3
geo	Contexte géographique (zone, proximité eau/habitat)	Muse Spark 1.3
research	Références vérifiables (jamais inventées)	Muse Spark 1.3
analysis	Synthèse + grille risque low→critical	Muse Spark 1.3
report	Sections + dashboard_hints pour l'UI	Muse Spark 1.3
router (Wonder)	Plan d'orchestration A2A	Muse Wonder 1.0

7 Dashboard temps réel

Le dashboard (*/dashboard*) est servi par FastAPI en HTML natif : 4 cartes stats, flux **LIVE RTSP — CAM 1**, panneau **AGENTS — LIVE** (6 agents + latences), **ALERTS LOG** et **SYSTEM STATUS**.

Interaction : **upload image/vidéo** (extraction 1 fps, max 4 frames) ou **Caméra réelle** (getUserMedia, 1,5 s). Les boîtes sont dessinées en DOM + SVG (polygone *fill 0,22*, couleurs par catégorie : *waste vert, plastic bleu, container jaune…*). Un **toggle ■ Son ON/OFF** contrôle l'alerte sonore.

Temps réel : WebSocket */ws/events* pousse *{type:event, event, alert, orchestration}* ; */ws/ingest/{source_id}* ingère le flux continu (réponses *queued* / *sample_skipped* à 1 fps, bypass si hint *waste*).

API principale

Route	Méthode	Description
/health	GET	status, version, env, counts, vision/router — sonde Docker & CI
/	GET	Message d'accueil + liens /docs /dashboard
/dashboard	GET	HTML temps réel (Live, boxes, alerts log)
/events	GET	Liste événements (?limit=50)
/events/{id}	GET	Détail événement
/alerts	GET	Liste alertes
/sources	GET	État sources vidéo
/ingest/image	POST	file JPEG/PNG + ?source_id=camera_001 → queued
/ingest/b64	POST	{source_id, b64} → queued
/ws/ingest/{id}	WS	Flux binaire/base64 continu
/ws/events	WS	Push temps réel vers dashboard

8 Qualité, Sécurité & CI/CD

Exigence : **un pipeline qui ne régresse pas**. La qualité est automatisée à chaque commit.

Ruff	**Mypy**	**Pytest**	**Coverage**	**Bandit**	**Semgrep**	**pip-audit**
lint + format S (bandit) inclus	64 fichiers 0 issue	28 tests 51% cov branch	html + term-missing seuil configurable	SAST Python 0 issue (8 nosec)	304 règles 0 finding	CVE scan skip-editable

CI/CD (.github/workflows/ci.yml, 4 jobs) : *lint → security (bandit/pip-audit/semgrep/ruff S) → tests (pytest --cov) → build Docker → push ghcr.io* sur *main*. *Makefile* expose *lint, format, type, test, cov, security-**. Pre-commit (ruff, mypy, bandit, semgrep, pip-audit) disponible.

Sécurité applicative : validation *source_id* regex, *max_image_bytes* 5 MB, sanitization prompt injection, CORS restrictif en prod (CORS_ORIGINS), secrets uniquement via env, logs sans secrets, *nosec* documentés pour les *try/except:pass* intentionnels.

Tests : unitaires (a2a, alerts, events, ingestion, vision), intégration (pipeline, main), contrat/détection (*test_detection_guarantees.py* : <1 min + boxes + son).

9 Déploiement & Exploitation

L'image Docker est **1-click** : stage *builder* (uv sync) → *runtime* (python:3.12-slim, user *app* non-root, HEALTHCHECK */health*, EXPOSE 8000). *docker-compose.yml* monte *app* + service *deepeval* (profile *eval*).

Plateformes : **Railway / Render** détectent le *Dockerfile* sans modif — il suffit de renseigner *MODEL_API_KEY, BASE_URL, VISION_PROVIDER*… et *PORT*. Image CI poussée vers *ghcr.io/<owner>/Eco-sentinel-Africa:latest*.

Exploitation : *GET /health* pour sondes, *store.count()* pour volumes, logs corrélés *correlation_id*, métriques *observability/metrics*. Observabilité prête pour OpenTelemetry/Prometheus.

Action	Commande
Dev	`uv sync --all-extras && uv run uvicorn eco_sentinel.api.app:app --reload --port 8000`
Prod Docker	`docker compose up --build -d → http://localhost:8000/health`
Tests + cov	`make test && make cov (ou uv run pytest --cov=src)`
Sécurité	`make security (bandit + pip-audit + semgrep + ruff S)`
Éval LLM	`docker compose --profile eval run --rm deepeval`

10 Résultats & Démonstration

Live public : **eco-sentinel-africa-latest.onrender.com/dashboard** • API */health* + */docs* (OpenAPI).

Indicateur	Valeur	Note
Latence vision→événement	< 50 ms mock / 2–4 s Meta	Mesurée en test_detection_guarantees
Précision signal high	NMS + filtre 0,45 + grille 0,78/0,90	Faux positifs mock 70 % → 14 %
Tests	28 passed	Ruff + Mypy 0 issue, Semgrep 0, Bandit 0
Couverture	51 %	Branch, html + term-missing
Ingestion	Queue 100, 1 fps, 5 MB max	Backpressure validée
Déploiement	Docker ready, CI 4 jobs	Render live vérifié

Démonstration jury (2 min) : 1) ouvrir le dashboard Render, 2) uploader une photo contenant *waste/plastic* (ou activer la webcam), 3) voir les boxes + l'alerte sonore + la carte agents se remplir en temps réel.

11 Limites, Éthique & Garanties

Nous affichons explicitement : **SAM détecte des objets, il ne prouve pas la toxicité chimique.** Les analyses agent sont tracées comme *hypothèses* avec incertitudes — jamais comme vérité terrain.

- Store in-memory (1000 events) → migration Postgres/S3/Redis prévue, sans changement d'interface.
- Mock tracé *simulated:true* uniquement si Meta indisponible — pas de silently fake.
- Couverture 51 % en croissance — les modules critiques (detector, ingestion, alerts) sont couverts en priorité.
- Données personnelles : aucune collecte — seules des images de déchets, anonymisables, avec rétention courte.
- Biais : entraînement SAM 3.1 généraliste — fine-tuning local prévu avec dataset ivoirien.

Éthique : l'outil **assiste** les agents municipaux, ne les remplace pas ; la décision d'intervention reste humaine, l'IA fournit la preuve visuelle et la priorisation.

12 Roadmap & Impact Afrique

De pilote Abidjan à réseau continental : chaque ville ajoute ses caméras, le même pipeline s'y branche.

Horizon	Livrable	Impact
T1 2027	Postgres + S3 + auth RBAC	Rétention & multi-utilisateurs
T2 2027	OpenTelemetry / Prometheus + Grafana	SLA & alerting ops
T2 2027	Dataset local + fine-tuning SAM	Précision waste Afrique +15 %
T3 2027	App mobile terrain (photo → ingestion)	Signalement citoyen géolocalisé
T4 2027	Fédération multi-villes (Abidjan → Dakar → Kigali)	Cartographie continentale

Impact attendu : -50 % de délais d'intervention, cartographie ouverte des points noirs, données pour politiques publiques (taxe plastique, tournées), emploi local (opérateurs sentinelles).

13 Équipe & Contact

Porteur	bony — essbony@outlook.fr — github.com/essbony/Eco-sentinel-Africa
Stack	Python 3.12 • FastAPI • Meta SAM 3.1 / Wonder / Spark • Docker • CI/CD
Live	eco-sentinel-africa-latest.onrender.com/dashboard • /health • /docs
Licence	MIT — réutilisable par collectivités et ONG

« Pipeline fiable, pas un chatbot. Une sentinelle qui voit, prouve et alerte — pour une Afrique plus propre. »

Merci au jury pour votre attention.

Annexe : structure, API et variables disponibles dans le README et le dossier technique remis en complément.